

# Концепт решения и архитектура

## Сервисы и базовые сущности

---

Любой запрос данных в системе проходит через сервисы проверки прав, после получения и диспетчеризации запроса в любом приложении запрос попадает в модуль проверки прав `PermissionChecker`.

`PermissionChecker`, в свою очередь, обращается за вычислением прав текущего запроса в модуль `PermissionCore`.

Для вычисления прав сервис должен получить данные о текущем пользователе и настройках его прав.

Получение прав может отличаться для разных сервисов:

- Есть основные сервисы (`online`, `standard`, `ext` и т.п.). Для основных внутри сервиса всегда есть дистрибутив, из которого мы можем получить все инсталляционные данные, например, роли. Все пользовательские настройки при этом сохраняются в БД в схеме клиента.
- Для прочих сервисов (например, `spp-admin` `tender-admin` `activity-admin`) механизм абсолютно другой: каждый прочий сервис при своем обновлении отправляет информацию о своих ролях в один сервис — сервис управления правами. Сервис управления правами складывает эту информацию в свою локальную БД.

Админка обращается именно к сервису управления правами и оттуда берет роли и папки.

Централизованное хранение и настройка прав сервисов облака, а так же проверка прав к участкам системы пользователей персональных групп осуществляется через подсистему — сервис [Управление правами \(rights-mng\)](#), а вся информация о пользователях получается на сервисе [Пользователи](#). В зависимости от конфигурации сервиса за данными настройки прав `PermissionCore` обращается в модуль `PermissionProviderDB`, если сконфигурировано с локальной базой данных, или в `PermissionProviderRemote`, если сконфигурирован с удаленным сервисом.

Для общего отображения данных всех сервисов используется интерфейс сервиса [Управление облаком](#).

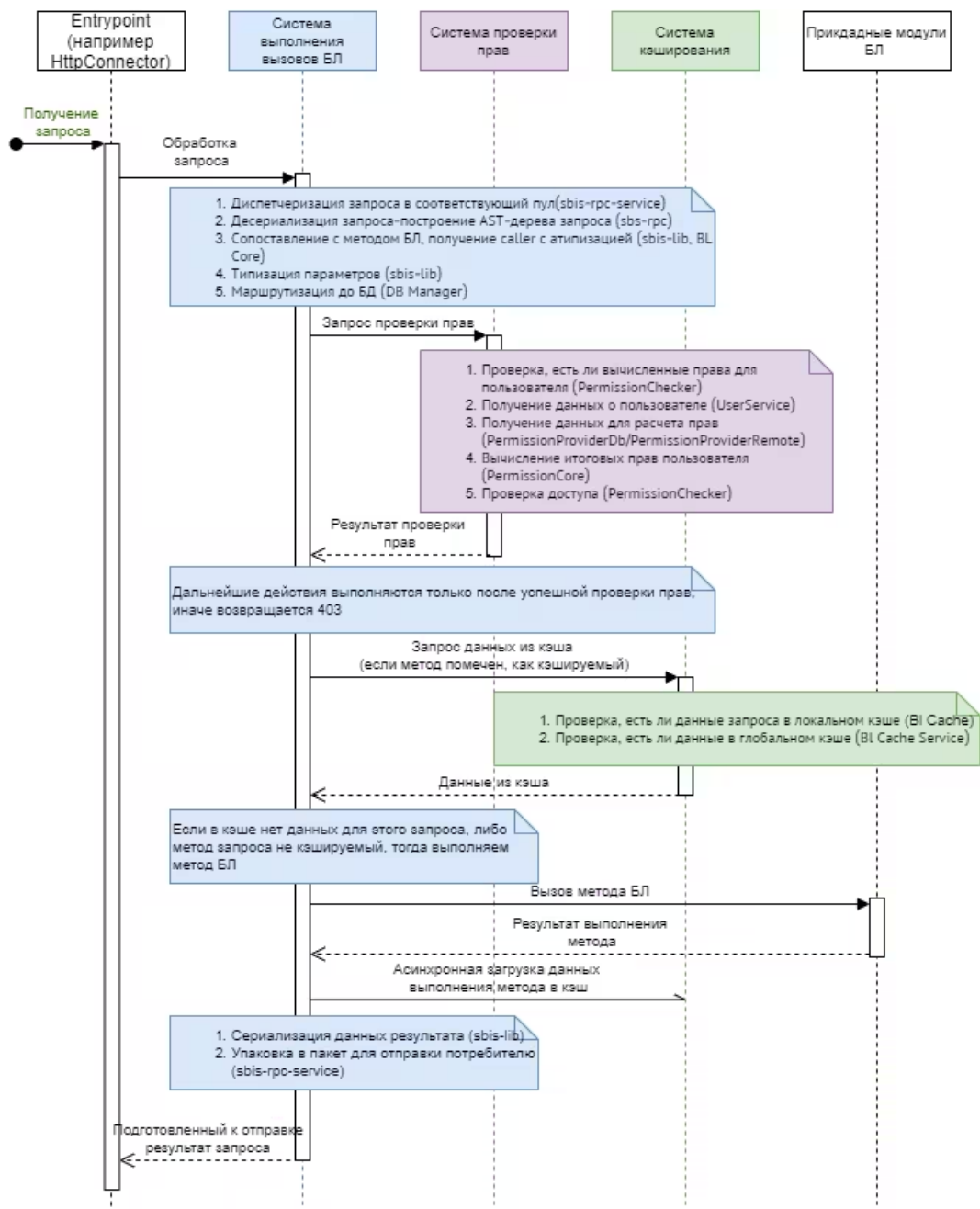


Рис 1. Flow общего прохождения запросов через систему прав

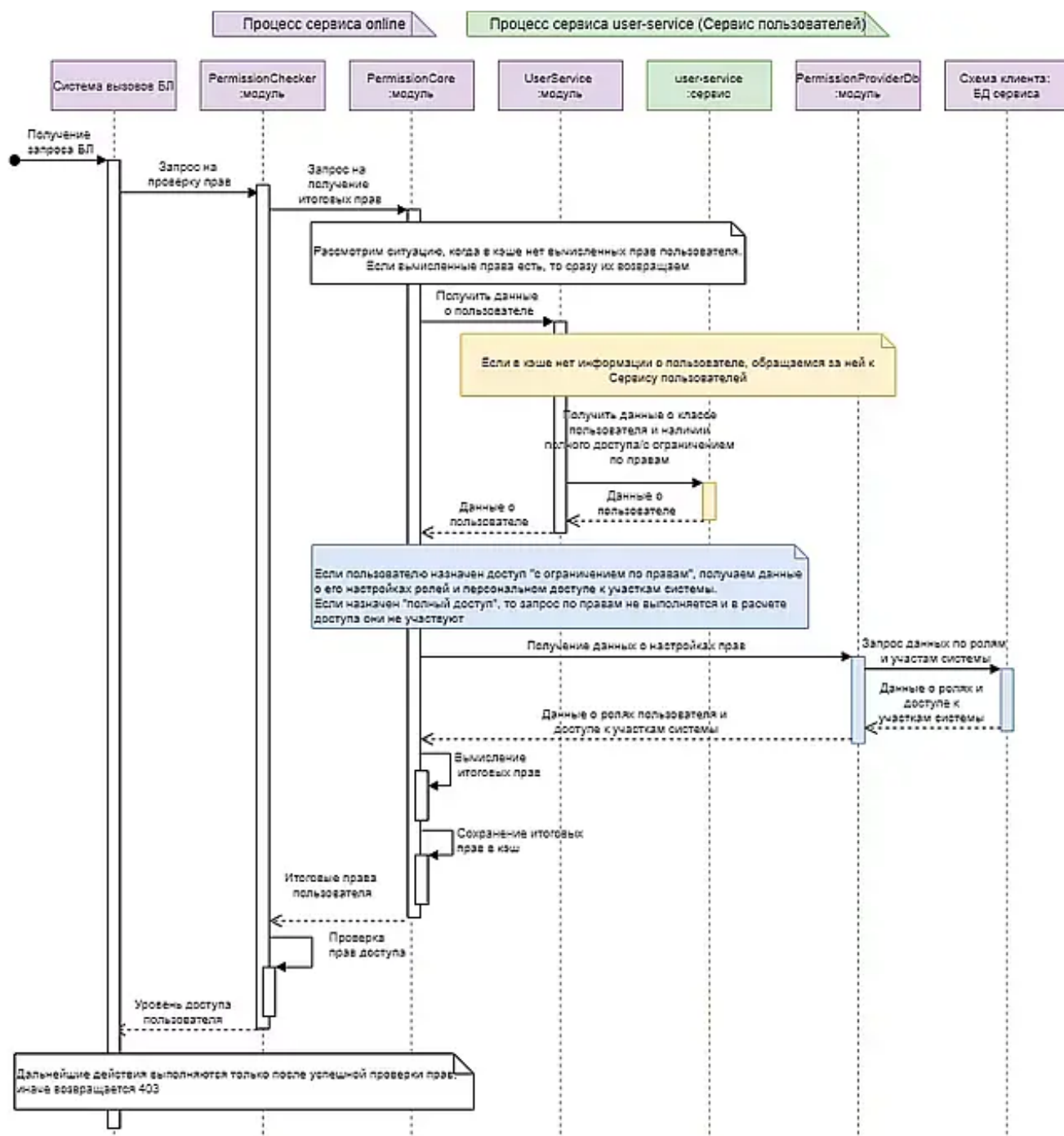


Рис 2. Процесс проверки прав для сервисов с локальной БД

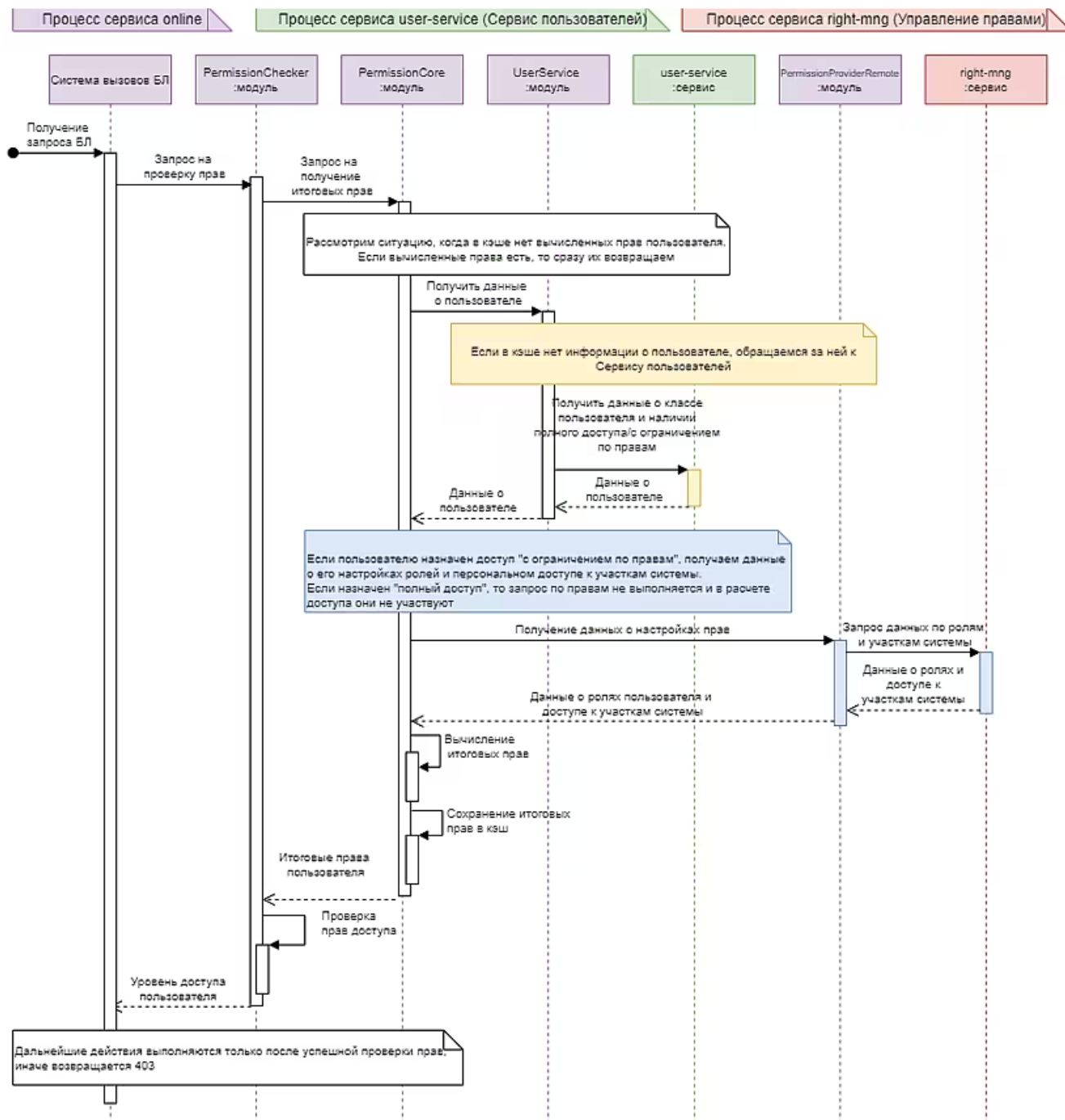


Рис 3. Процесс проверки прав для сервисов с удаленной проверкой прав

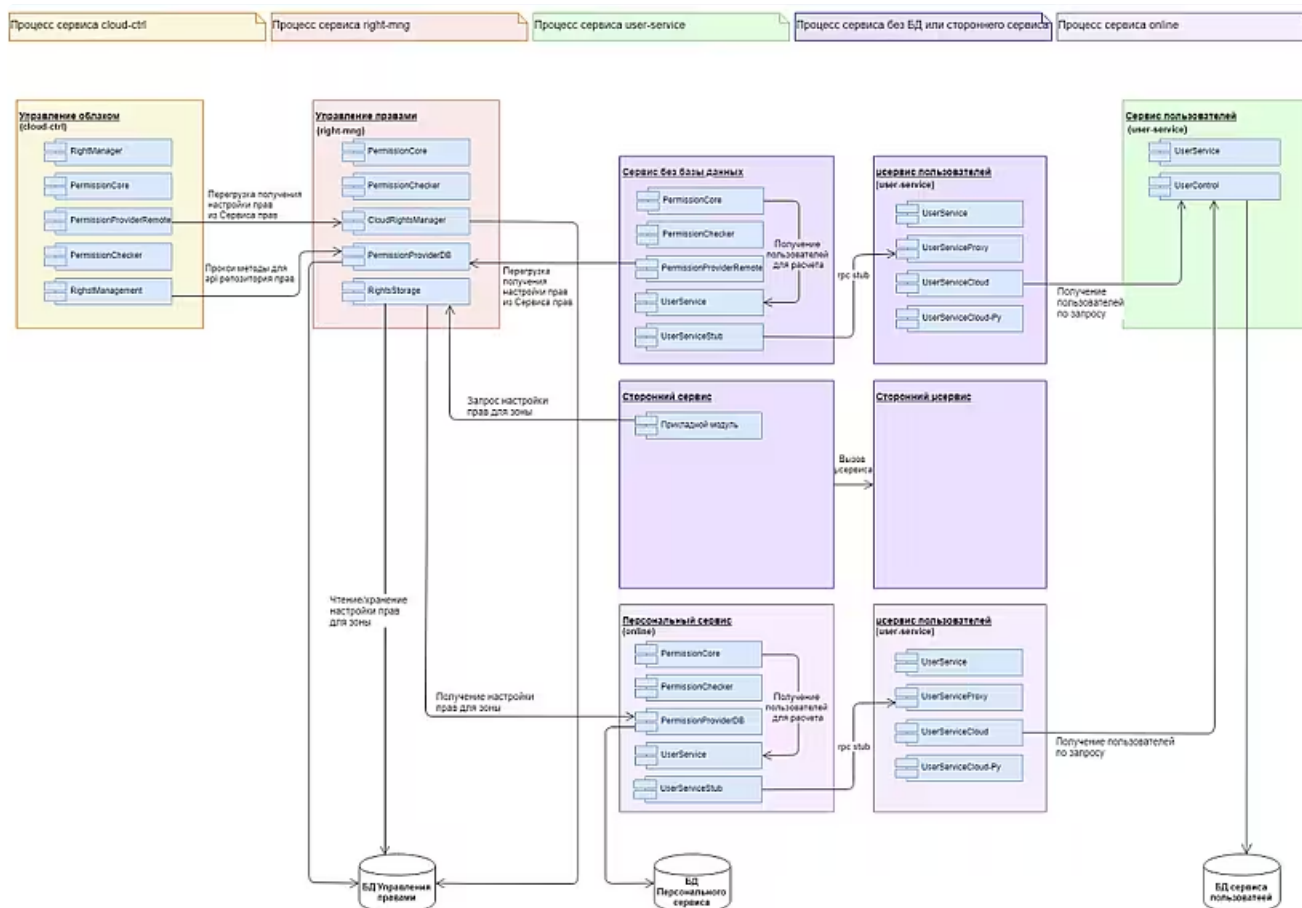


Рис 4. Архитектура сервисов системы прав

## Подсистема расчета прав в модуле Permission Core

Основные абстракции системы прав определяются в модуле Permission Core (библиотека [sbis-permission-core](#)).

Основные типы данных:

- **Action** — соответствует зоне доступа, хранит все ее характеристики, прочитанные из метаданных
- **ActionGraph** — граф зон доступа, содержит сами зоны и отношения связей (вложенности) между ними.
- **RoleView** — соответствует роли. Не хранит свои данные, а предоставляет доступ к данным из графа прав какого-либо типа по ссылке. Дает доступ только к данным самой роли, но не к списку включенных в нее зон доступа.
- **RoleList** - объект для хранения набора данных списка ролей. Организован по принципу [structure of arrays](#). Применяется для имплементации графов прав разного уровня.
- **Графы прав** — вспомогательные структуры данных, используемые для представления отношений между ролями и участками системы на разных уровнях.

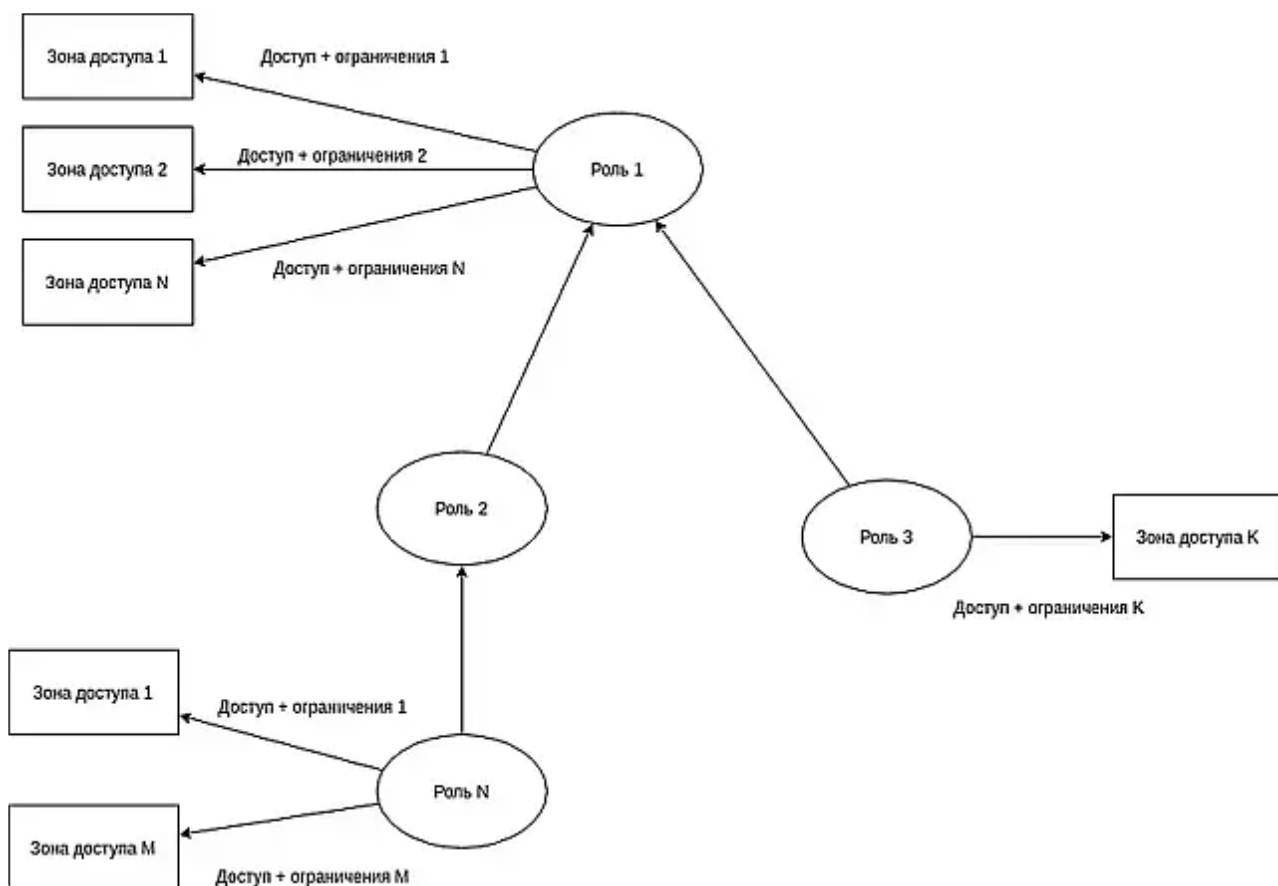


Рис. 5. Граф прав

**SystemRightsGraph** — системный граф прав. Хранит инсталляционные роли и отношения между ними (а также отношения между ролями и зонами доступа) для одного из классов пользователей. Набор системных графов прав (по одному на класс пользователя) создается при старте приложения на основе инсталляционных данных (rix-файлов) при помощи экземпляра класса SystemRightsGraphBuilder. Будучи однажды созданным, объект данного класса уже не может быть изменен.

**ClientRightsGraph** — граф прав клиента. Хранит роли и отношения между ними (а также отношения между ролями и зонами доступа), актуальные в рамках одного клиента. Строится на основе системного графа прав для соответствующего класса пользователей. По сути, представляет собою «патч» на системный граф. Может содержать либо новые вершины (собственно клиентские роли), либо «перекрытые» инсталляционные роли. В случае перекрытия какой-либо системной роли в клиентский граф также вносятся все роли, которые берут права с перекрытой (наследуют/включают ее прямо или опосредованно). Если у клиента перекрывается роль «Пользователь системы», то фактически перекрытыми становятся все не служебные системные роли. Экземпляр данного класса создается при помощи экземпляра класса ClientRightsGraphBuilder. Будучи однажды созданным, объект данного класса уже не может быть изменен.

**UserRightsGraph** — граф прав одного пользователя. Как правило, хранит одну персональную роль пользователя. По сути является «патчем» на клиентский граф прав. Экземпляр данного класса создается при помощи экземпляра класса UserRightsGraphBuilder. Будучи однажды созданным, объект данного класса уже не может быть изменен.

**SystemRightsGraphBuilder** — «строитель» системных графов прав. Имеет API для добавления ролей, связей ролей друг с другом, а также связей между ролями и зонами доступа для разных классов пользователей. По окончании добавления данных после вызова метода Build() порождает набор системных графов прав.

**ClientRightsGraphBuilder** — «строитель» клиентского графа прав (поверх системного).

**UserRightsGraphBuilder** — «строитель» пользовательского графа прав (поверх клиентского).

**Матрицы прав** — вспомогательные структуры данных, применяемые для расчета прав конкретной роли на конкретную зону доступа. «Строками» и «столбцами» матриц прав являются соответственно роли и зоны доступа, а ячейками — права данной роли на данную зону доступа. Матрицы строятся на основе графов прав и графа зон доступа.

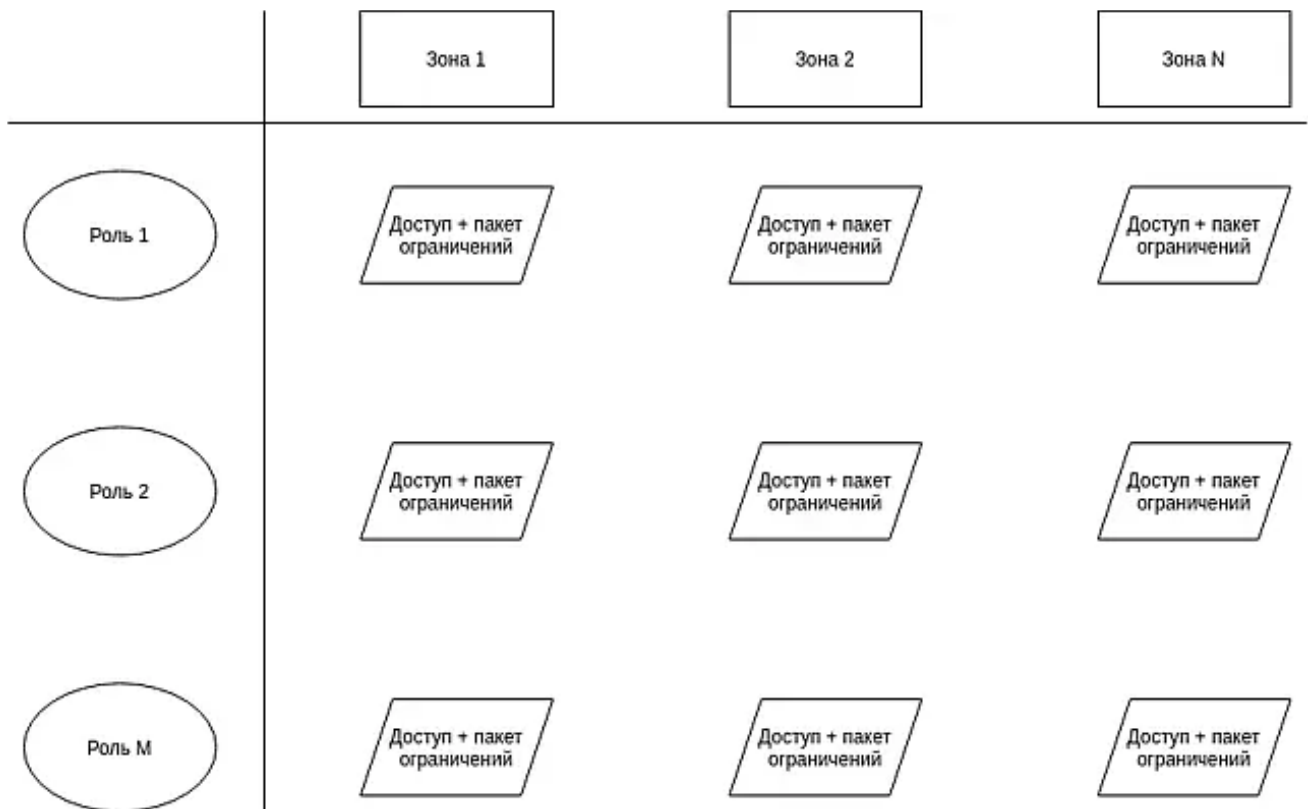


Рис. 6. Матрица прав

**SystemMatrix** — системная матрица прав. Строится при старте приложения на основе системного графа прав (по одной матрице на класс пользователей).

**ClientMarix** — клиентская матрица прав. Строится на основе клиентского графа прав поверх системной матрицы для соответствующего класса пользователей.

**UserMatrix** — пользовательская матрица прав. Строится на основе пользовательского графа прав поверх клиентской матрицы.

**Entry** — ячейка матрицы прав. Хранит три атрибута: доступ роли к участку, способ предоставления этого доступа (или «почему мы решили, что доступ именно такой»), пакет значений ограничений по областям видимости.

Для классов матриц реализованы кастомные стратегии управления памятью, за которые отвечает класс **EntryStorage**.

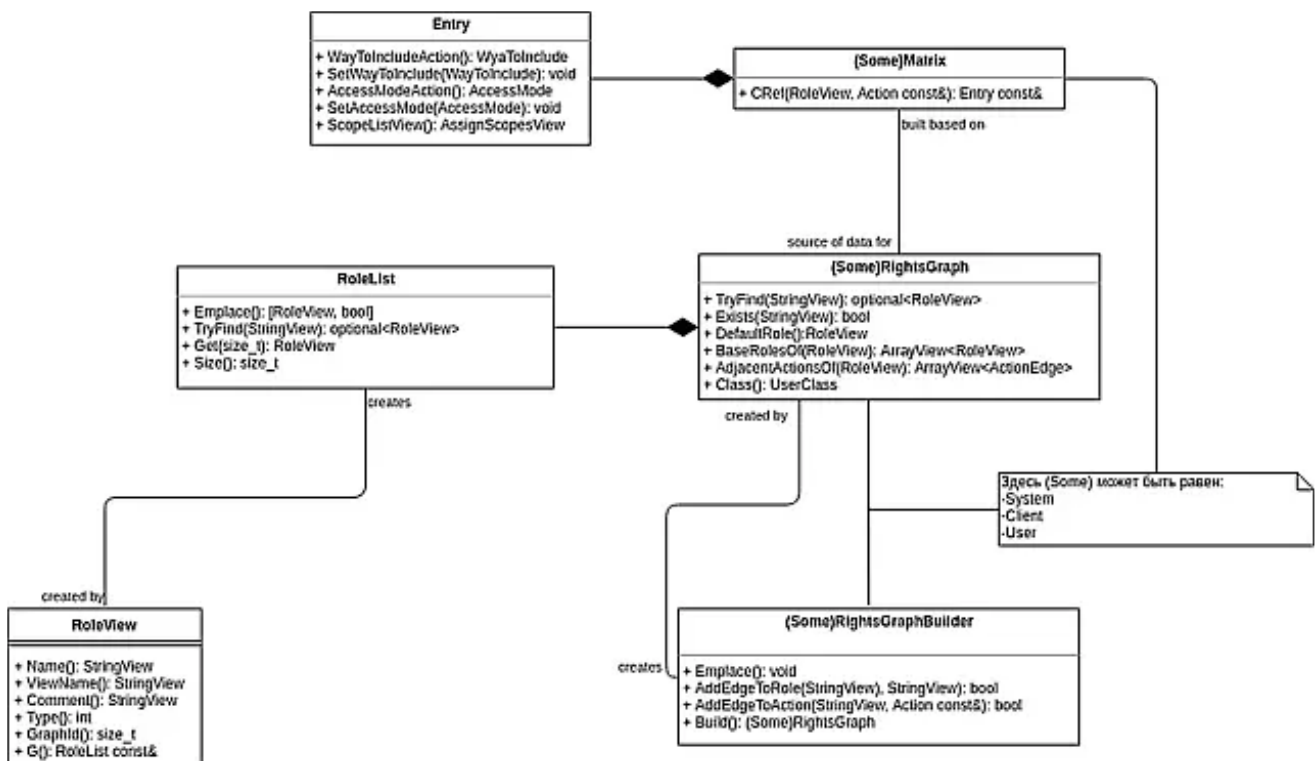


Рис. 7. Классы модуля Permission Core



## Подсистема источника прав

Для расчета прав в модуле Permission Core необходим источник данных, им может быть удаленный сервис или БД текущего сервиса. Технически источник данных – это реализация интерфейса PermissionDataProvider. Для работы триггера и базовых БЛ методов проверки прав одного пользователя достаточно реализации получения данных только по одному пользователю.

### EmployeeRightsPermissionDataProvider

Модуль Employee Rights описывающий права сотрудников предоставляет реализацию с учетом возможности назначения ролей на юридические и управленческие подразделения. Более подробно о связи подразделений, сотрудников и ролей в [базе данных системы прав](#).

Технически привязка должностей и подразделений к ролям реализована по разному:

**На подразделение** можно назначить несколько системных или пользовательских ролей через таблицу RoleParticipant.

Роли подразделений учитываются на пользователях опосредованно, при запросе прав берутся роли назначенные на все их подразделения по иерархическому принципу.

**На должность** роль не назначается. При создании привязки формируется уникальная пользовательская роль с аналогичным названием и ссылкой (поле PositionId) в таблице RoleDescription. Роль для должности может быть создана только одна. Создание/удаление роли может совершено через API Системы прав, либо через открытое API.

Назначение/снятие должностной роли на/с исполнителя происходит напрямую в таблице RoleParticipant по прикладному событию. Описание событий в [юридической](#) и [управленческой](#) структурах.

Для поддержания консистентности данных при возникновении события назначения должности на сотрудника, его должностные роли актуализируются целиком. Данные детализирующие изменения (напр. какая должность назначена) не используются. Важно чтобы обработка события происходила в той же транзакции после изменений в прикладных таблицах.

В юридической структуре есть время назначения и снятия должности. По этой причине актуализация должностных ролей может быть отложена, по средством создания соответствующей задачи вызова метода PositionRole.DelayedUpdatePosRolesHandler на сервис [task-scheduler](#).

## PermissionManager

До недавнего времени ограничений можно было получать либо через методы бизнес-логики, например, CheckRights.AccessAreaRestrictions, либо через sbis.Session.Rights(). Оба варианта не позволяют возвращать сложные типы и требуют расчета полного множества доступных объектов, поэтому реализован отдельный класс для работы с системой прав.

```
1 class PermissionManager {
2     /// Получить экземпляр PermissionManager
3     static PermissionManager& Instance();
4     /// Получить значение сложного ограничения для текущего метода и
    текущего пользователя
5     template< typename T > std::unique_ptr< IConstraintValue< T > >
    GetConstraintForCurrentMethod( StringView const constraint_id );
6     /// Получить значение сложного ограничения для текущего пользователя
    по указанным участкам template<
7     typename T > std::unique_ptr< IConstraintValue< T > >
    GetConstraintForAreas( StringView const constraint_id, ArrayString
    const& areas );
8     /// Получить значение простого ограничения для текущего метода и
    текущего пользователя
9     /// true - разрешено действие, соответствующее ограничению
10    /// false - запрещено действие, соответствующее ограничению bool
11    GetBoolConstraintForCurrentMethod( StringView const constraint_id );
12    /// Получить значение простого ограничения для текущего пользователя
    по указанным участкам
13    /// true - действие, соответствующее ограничению Разрешено
14    /// false - действие, соответствующее ограничению Запрещено bool
15    GetBoolConstraintForAreas( StringView const constraint_id );
16    /// Регистрация ограничения
17    template< typename T > void RegisterConstraint( StringView const
```



```

18     constraint_id, ConstraintCreator* creator );
    }

```

Класс доступен в [C++](#) и в [Python](#).

## IConstraintValue

Для работы через PermissionManager используется шаблонный интерфейс для значения ограничения.

```

1  class IConstraintValueBase {
2  public:
3  virtual ~IConstraintValueBase(){}
4  }
5  template< typename T >
6  class IConstraintValue : public IConstraintValueBase
7  {
8  public:
9      /// Получить все множество доступных объектов с разворотом
10     /// Привычный формат, ровно такой же, как был в объекте CheckRights
или в sbis.Session.Rights()
11     virtual boost::optional< OptArray< T > > GetAllAccessibleIds() = 0;
12     /// Получить множество доступных объектов без разворота
13     /// По сути получить ограничение так, как его видит пользователь.
14     /// Если задано "Все разрешено кроме Платформа", то будем возвращен
Record с полями Ids: None (все разрешено) и Excluded:
[%ИдентификаторПлатформа%]
15     virtual boost::optional< Record > GetAccessibleSet( bool
resolve_flags ) = 0;
16     /// Получить по массиву идентификаторов список разрешенных
объектов.
17     virtual OptArray< T > GetAccessibleIds( OptArray< T > const& ) = 0;
18     /// Доступен ли конкретный объект
19     virtual bool IsObjectAccessible( T const& id ) = 0;
20 }

```

Прикладной разработчик для своего ограничения делает собственную реализацию на C++.

```

1  class MyConstraintValue: public IConstraintValue< T >

```

Также ограничение может быть реализовано на Python с аналогичной сигнатурой.

```

1  object IConstraintValue(object):
2      def GetAllAccessibleIds(self) -> list:
3          """Получить все множество доступных объектов с разворотом"""
4          pass
5      def GetAccessibleSet(self, resolve_flags: bool) -> sbis.Record:
6          """Получить множество доступных объектов без разворота"""
7          pass
8      def GetAccessibleIds(self, ids: list) -> list:
9          """Получить по массиву идентификаторов список разрешенных
объектов"""
10         pass
11     def IsObjectAccessible(self, object_id: object) -> bool:
12         """Доступен ли конкретный объект"""
13         pass

```

Для доступа из одного языка к ограничению, реализованном на другом языке, реализовано два класса обертки PythonConstraintValue и CppConstraintValue.

Кроме того, IConstraintValue - это всего лишь интерфейс с минимальным набором методов. Он может быть расширен для конкретного типа ограничений. В частности у StaffConstraintValue (наследник IConstraintValue для работы с ограничениями по подразделениям и по сотрудникам) есть метод SmartGetAllIds, который возвращает все множество разрешенных или запрещенных объектов, в зависимости от того, какое из них меньше.

## Регистрация ограничений

Соответствие между ограничением и конкретной реализацией IConstraintValue устанавливается на этапе загрузки модулей в обработчике OnEndAllLoadModules посредством вызова PermissionManager::RegisterConstraint.

Поведение RegisterConstraint несколько отличается в C++ и в Python. Для Python принимается класс значения ограничения, для создания конкретного значения вызывается конструктор указанного класса с передачей идентификатора пользователя и значения ограничения в виде Record (это те же аргументы, что сейчас передаются в функцию области видимости). Для C++ в RegisterConstraint принимается функтор, который по тем же аргументам должен вернуть

```
std::unique_ptr< IConstraintValue< T > >
```

В рамках одного языка для одного ограничения можно зарегистрировать только одну реализацию. Последующие вызовы будут перекрывать ранее выполненную реализацию. Для ограничения может быть зарегистрирована одна реализация на каждом поддерживаемом языке.

При создании значения ограничения используется реализация текущего языка, если такая существует. Если на текущем языке реализации нет, то используется реализация на другом языке через обертки PythonConstraintValue и CppConstraintValue. Т.о. используется "ближайшая" для потребителя реализация, кроме того это позволяет расширить интерфейс IConstraintValue для конкретного ограничения, если в этом будет необходимость.

## DefaultConstraintValue

Чтобы можно было сразу пользоваться новым API для всех ограничений и чтобы не заставлять всех ответственных за ограничения делать однотипные классы, создан DefaultConstraintValue (наследник IConstraintValue). Эта реализация используется, когда нет ни одной другой реализации значения конкретного ограничения.

Внутри DefaultConstraintValue происходит вызов области видимости и получение всего множества доступных объектов. Этот список запоминается и используется для ответов в методах интерфейса IConstraintValue.

## Переходный период

Т.к. написано много кода с использованием sbis.Session.Rights(), то мы не можем сразу от него отказаться и перейти на PermissionManager, поэтому вычисление ограничений происходит "лениво", т.е. ограничения вычисляются при первом обращении к sbis.Session.Rights().

Это с одной стороны позволяет переходить на новое API постепенно, с другой стороны в конкретных проблемных сценариях сможем получить ускорения сразу.

## Ограничения сотрудников по умолчанию

Ограничения сотрудников по умолчанию настраиваются на уровне дистрибутива. На текущий момент по умолчанию действует ограничение "По подразделениям" со значением "Свой офис". Клиент может изменить эти значения в карточке роли "Пользователь системы".

[Подробнее описание бизнес-процесса](#)

## Взаимосвязь сущностей системы прав, поставляемых в SDK и справочников метаданных Genie

---

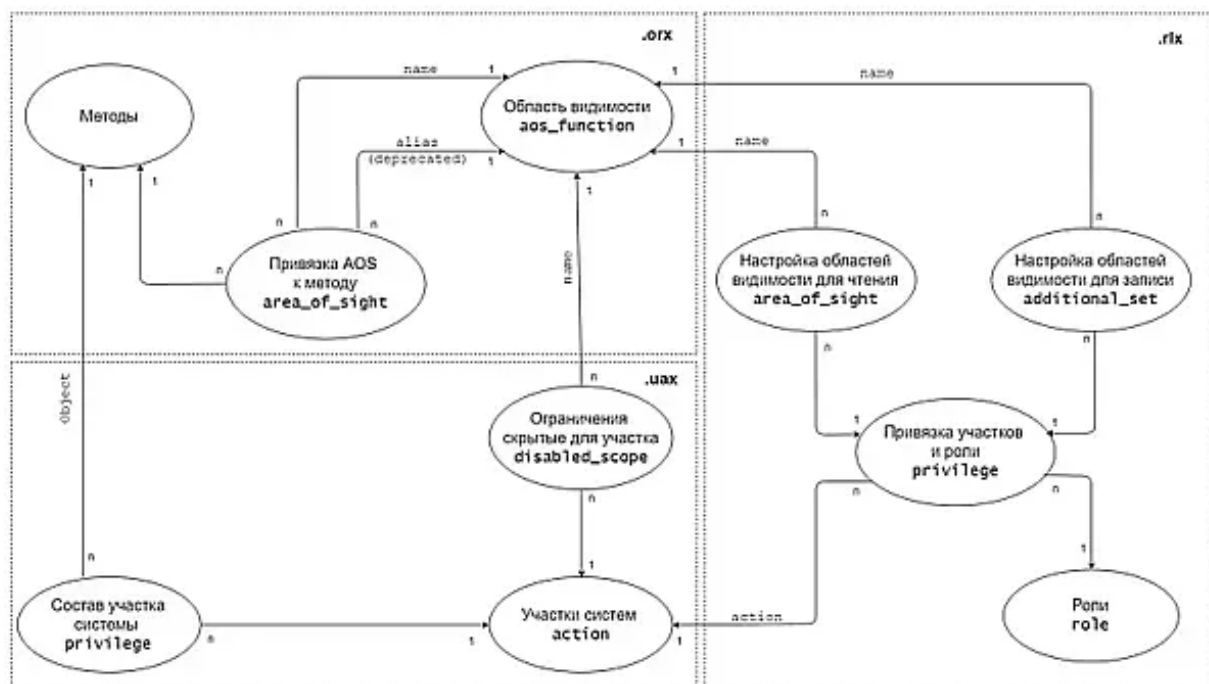


Рис. 8. Схема взаимосвязи сущностей системы прав